Reinforcement Learning

Variational Inference and Control as Inference

SJTU

Outline

1. Latent Variable Models
2. Training Latent Variable Models
3. Variational Inference
4. Amortized Variational Inference
5. Variational Autoencoders
6. Control as Inference
7. Summary

Probabilistic Models

Two types of probabilistic models:

- **Density model** $p(x)$: model the distribution of data
  - Example: density estimation over images, states
- **Conditional model** $p(y|x)$: predict output given input
  - Example: policy $\pi_\theta(\mathbf{a}|\mathbf{s})$, regression $p(y|x)$

Both are fundamental building blocks in RL:

- Policies, dynamics models, reward models, value functions

Latent Variable Models

Key idea: represent complex distributions as *mixtures* of simple ones

- Density model with latent z:

$$p(x) = \sum_z p(x|z)\, p(z) \quad \text{(discrete)} \qquad p(x) = \int p(x|z)\, p(z)\, dz \quad \text{(continuous)}$$

- Conditional model with latent z:

$$p(y|x) = \sum_z p(y|x, z)\, p(z)$$

- Each value of z selects a "mixture element"

Simple components $\rightarrow$ complex marginal distribution

Latent Variable Models in General

$$p(x) = \int p(x|z)\, p(z)\, dz$$

- $p(z)$: "easy" prior (e.g., Gaussian)
- $p(x|z)$: "easy" conditional (e.g., conditional Gaussian)
- $p(x)$: complex, multimodal marginal

Neural network parameterization:

$$p(x|z) = \mathcal{N}\big(\mu_{\text{nn}}(z),\, \sigma_{\text{nn}}(z)\big)$$

- Network maps z to parameters of $p(x|z)$
- Infinite mixture of Gaussians

Latent Variable Models in RL

Multi-modal policies:

- Conditional latent variable model
- $p(\mathbf{a}|\mathbf{s}, z)$ with $z \sim \mathcal{N}(0, \mathbf{I})$
- Models multiple strategies for the same state

Model-based RL (state space models):

- Latent states $\mathbf{s}_{1:H}$ form structured latent space
- $p(\mathbf{o}_t|\mathbf{s}_t)$: observation model
- $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$: transition model
- $p(\mathbf{s}_1)$: initial state

How Do We Train Latent Variable Models?

- The model: $p_\theta(x)$
- The data: $\mathcal{D} = \{x_1, x_2, \ldots, x_N\}$
- Maximum likelihood:

$$\theta \leftarrow \arg\max_\theta \frac{1}{N} \sum_i \log p_\theta(x_i)$$

But for latent variable models:

$$\theta \leftarrow \arg\max_\theta \frac{1}{N} \sum_i \log \left(\int p_\theta(x_i|z)\, p(z)\, dz \right)$$

The integral inside the log makes this **completely intractable**!

Estimating the Log-Likelihood

Alternative: use the *expected* log-likelihood:

$$\theta \leftarrow \arg\max_{\theta} \frac{1}{N} \sum_{i} E_{z \sim p(z|x_i)} \big[\log p_\theta(x_i, z) \big]$$

- Intuition: "guess" the most likely z given x_i, use $p(z|x_i)$
- But... how do we calculate $p(z|x_i)$?

This is called **probabilistic inference** — the central challenge

The Variational Approximation

How do we calculate $p(z|x_i)$? **Approximate** with $q_i(z) = \mathcal{N}(\mu_i, \sigma_i)$

We can bound $\log p(x_i)$:

$$\begin{aligned}
\log p(x_i) &= \log \int_z p(x_i|z)\, p(z)\, dz \\
&= \log \int_z p(x_i|z)\, p(z)\, \frac{q_i(z)}{q_i(z)}\, dz \\
&= \log E_{z\sim q_i(z)}\left[\frac{p(x_i|z)\, p(z)}{q_i(z)}\right] \\
&\geq E_{z\sim q_i(z)}\left[\log \frac{p(x_i|z)\, p(z)}{q_i(z)}\right] \quad \text{(Jensen's inequality: } \log E[y] \geq E[\log y]\text{)}
\end{aligned}$$

The Evidence Lower Bound (ELBO)

Expanding the bound:

$$\log p(x_i) \geq \underbrace{E_{z\sim q_i(z)}\big[\log p(x_i|z) + \log p(z)\big] + \mathcal{H}(q_i)}_{\mathcal{L}_i(p,q_i) \text{ — the ELBO}}$$

- $E_{z\sim q_i}[\log p(x_i|z)]$: expected reconstruction quality
- $E_{z\sim q_i}[\log p(z)]$: closeness of q_i to the prior
- $\mathcal{H}(q_i) = -E_{z\sim q_i}[\log q_i(z)]$: entropy of q_i (encourages spread)

Maximizing the ELBO $\mathcal{L}_i(p, q_i)$ maximizes a lower bound on $\log p(x_i)$

A Brief Aside: Entropy

Entropy:

$$\mathcal{H}(p) = -E_{x\sim p(x)}[\log p(x)] = -\int_x p(x) \log p(x)\, dx$$

- **Intuition 1:** how *random* is the random variable?
- **Intuition 2:** how large is the log probability in expectation *under itself*?
- High entropy $\Leftrightarrow$ spread-out, uniform-like distribution
- Low entropy $\Leftrightarrow$ peaked, concentrated distribution

A Brief Aside: KL-Divergence

KL-Divergence:

$$D_{\mathrm{KL}}(q\|p) = E_{x\sim q(x)}\left[\log \frac{q(x)}{p(x)}\right] = E_{x\sim q}[\log q(x)] - E_{x\sim q}[\log p(x)]$$

- **Intuition 1:** how *different* are two distributions?
- **Intuition 2:** *negative* expected log-probability of p under q, minus the entropy of q: $-E_{x\sim q}[\log p(x)] - \mathcal{H}(q)$
- $D_{\mathrm{KL}}(q\|p) \geq 0$ always (Gibbs' inequality)
- $D_{\mathrm{KL}}(q\|p) = 0 \Leftrightarrow q = p$
- **Not symmetric:** $D_{\mathrm{KL}}(q\|p) \neq D_{\mathrm{KL}}(p\|q)$ in general

The Variational Identity

Key result: we can decompose the log-evidence *exactly*:

$$D_{\mathrm{KL}}\big(q_i(z)\|p(z|x_i)\big) = -E_{z\sim q_i}\big[\log p(x_i|z) + \log p(z)\big] - \mathcal{H}(q_i) + \log p(x_i)$$

Rearranging:

$$\boxed{\log p(x_i) = D_{\mathrm{KL}}\big(q_i(z)\|p(z|x_i)\big) + \mathcal{L}_i(p, q_i)}$$

- Since $D_{\mathrm{KL}} \geq 0$: $\quad \log p(x_i) \geq \mathcal{L}_i(p, q_i) \quad$ (confirms ELBO is a lower bound)
- Maximizing $\mathcal{L}_i$ w.r.t. $q_i \Rightarrow$ minimizes $D_{\mathrm{KL}}(q_i\|p(z|x_i))$
- At optimum: $q_i^* = p(z|x_i)$ and the bound is tight

Variational Inference Algorithm

Replace intractable MLE with ELBO maximization:

$$\theta \leftarrow \arg\max_{\theta} \frac{1}{N} \sum_{i} \mathcal{L}_i(p, q_i)$$

For each x_i (or mini-batch):

1. **Compute** $\nabla_\theta \mathcal{L}_i(p, q_i)$**:**
   - Sample $z \sim q_i(z)$
   - $\nabla_\theta \mathcal{L}_i(p, q_i) \approx \nabla_\theta \log p_\theta(x_i|z)$
2. Update $\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}_i(p, q_i)$
3. **Update** q_i**:** gradient ascent on μ_i, σ_i using $\nabla_{\mu_i} \mathcal{L}_i$, $\nabla_{\sigma_i} \mathcal{L}_i$

Let $q_i(z) = \mathcal{N}(\mu_i, \sigma_i)$

The Scalability Problem

How many parameters are there?

$$|\theta| + (|\mu_i| + |\sigma_i|) \times N$$

- Model parameters θ: shared across all data
- Variational parameters (μ_i, σ_i): **one set per data point!**
- For $N = 10^6$ images with d-dimensional z: $2dN$ extra parameters ($d = 64$: 1.28×10^8)

Next: deep learning makes this tractable via **amortization**

From Per-Sample to Amortized Inference

Observation: $q_i(z)$ should approximate $p(z|x_i)$

Idea: learn a *network* $q_\phi(z|x) \approx p(z|x)$

- **Before (VI):** separate $q_i(z) = \mathcal{N}(\mu_i, \sigma_i)$ for each x_i
- **After (amortized VI):** single network $q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$
- Parameters: $|\theta| + |\phi|$ instead of $|\theta| + (|\mu_i| + |\sigma_i|) \times N$

Two networks:

- **Decoder** (generative model): $z \to p_\theta(x|z)$
- **Encoder** (inference network): $x \to q_\phi(z|x)$

Amortized VI: The Algorithm

ELBO with amortized inference:

$$\log p(x_i) \geq E_{z \sim q_\phi(z|x_i)}\big[\log p_\theta(x_i|z) + \log p(z)\big] + \mathcal{H}\big(q_\phi(z|x_i)\big)$$

For each x_i (or mini-batch):

1. Sample $z \sim q_\phi(z|x_i)$
2. $\nabla_\theta \mathcal{L} \approx \nabla_\theta \log p_\theta(x_i|z)$ (update decoder)
3. $\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}$
4. $\phi \leftarrow \phi + \alpha \nabla_\phi \mathcal{L}$ (update encoder — but how?)

Problem: $\nabla_\phi \mathcal{L}$ requires differentiating through $z \sim q_\phi(z|x_i)$

Policy Gradient Approach

Define $J(\phi) = E_{z \sim q_\phi(z|x_i)}[r(x_i, z)]$ where $r(x_i, z) = \log p_\theta(x_i|z) + \log p(z)$

(The entropy term $\mathcal{H}(q_\phi(z|x_i))$ of $\mathcal{L}_i$ is left out of J: it has a closed form for Gaussian q_ϕ and is differentiated directly.)

Policy gradient (REINFORCE-style):

$$\nabla J(\phi) \approx \frac{1}{M} \sum_j \nabla_\phi \log q_\phi(z_j|x_i)\, r(x_i, z_j)$$

✓ Can handle both discrete and continuous latent variables

× High variance, requires many samples & small learning rates

The Reparameterization Trick

Key insight: rewrite the sampling so the randomness is *independent* of ϕ

$$z = \mu_\phi(x) + \epsilon\,\sigma_\phi(x), \qquad \epsilon \sim \mathcal{N}(0,1)$$

Then:

$$\begin{aligned} J(\phi) &= E_{z\sim q_\phi(z|x_i)}[r(x_i, z)] \\ &= E_{\epsilon\sim\mathcal{N}(0,1)}\big[r\big(x_i,\ \mu_\phi(x_i) + \epsilon\,\sigma_\phi(x_i)\big)\big] \end{aligned}$$

Estimating $\nabla_\phi J(\phi)$:

$$\nabla_\phi J(\phi) \approx \frac{1}{M}\sum_j \nabla_\phi r\big(x_i,\ \mu_\phi(x_i) + \epsilon_j\,\sigma_\phi(x_i)\big)$$

- ✓ Low variance
- ✓ Simple to implement (autodiff!)
- × Only continuous z

Reparameterization Trick vs. Policy Gradient

Policy gradient:

$$\nabla J(\phi) \approx \frac{1}{M} \sum_j \nabla_\phi \log q_\phi(z_j|x_i)\, r(x_i, z_j)$$

- Discrete **and** continuous z
- High variance
- Requires many samples, small LR

Reparameterization trick:

$$\nabla_\phi J(\phi) \approx \frac{1}{M} \sum_j \nabla_\phi r(x_i, \mu_\phi(x_i) + \epsilon_j \sigma_\phi(x_i))$$

- Only continuous z
- Low variance
- Simple to implement (PyTorch/JAX)

The VAE Objective

Rewrite the ELBO in a more convenient form:

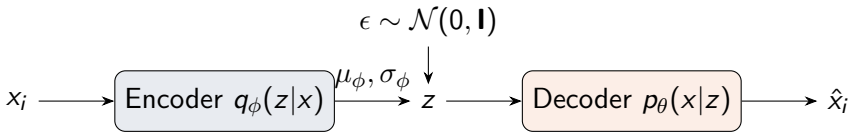
$$\begin{aligned}\mathcal{L}_i &= E_{z\sim q_\phi(z|x_i)}\big[\log p_\theta(x_i|z) + \log p(z)\big] + \mathcal{H}\big(q_\phi(z|x_i)\big) \\ &= E_{z\sim q_\phi(z|x_i)}\big[\log p_\theta(x_i|z)\big] - D_{\mathrm{KL}}\big(q_\phi(z|x_i)\|p(z)\big)\end{aligned}$$

- $E[\log p_\theta(x_i|z)]$: **reconstruction term** (decode z back to x)
- $D_{\mathrm{KL}}(q_\phi(z|x_i)\|p(z))$: **regularization** (keep encoder close to prior)
- KL between two Gaussians has **closed-form** solution

Training:

$$\max_{\theta,\phi} \frac{1}{N}\sum_i \log p_\theta(x_i|\mu_\phi(x_i) + \epsilon\,\sigma_\phi(x_i)) - D_{\mathrm{KL}}\big(q_\phi(z|x_i)\|p(z)\big)$$

The Variational Autoencoder



- **Encoder:** $q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$
- **Decoder:** $p_\theta(x|z) = \mathcal{N}(\mu_\theta(z), \sigma_\theta(z))$
- **Prior:** $p(z) = \mathcal{N}(0, \mathbf{I})$

Sampling: $z \sim p(z)$, then $x \sim p_\theta(x|z)$

Example Applications: Representation Learning for RL

Idea: z is a *representation* of state s

Algorithm:

1. Train VAE on states in replay buffer $\mathcal{R}$
2. Run RL using z as the state instead of s

Sample algorithm (online):

1. Collect transition (s, a, s', r), add to $\mathcal{R}$
2. Update $p_\theta(s|z)$ and $q_\phi(z|s)$ with batch from $\mathcal{R}$
3. Update $Q(z, a)$ with batch from $\mathcal{R}$

Compact latent space $\rightarrow$ better generalization for RL

Conditional Models and Multimodal Imitation

Conditional VAE — generate y conditioned on x:

$$\mathcal{L}_i = E_{z\sim q_\phi(z|x_i,y_i)}\big[\log p_\theta(y_i|x_i,z) + \log p(z|x_i)\big] + \mathcal{H}\big(q_\phi(z|x_i,y_i)\big)$$

- Encoder sees both x_i and y_i; decoder generates y_i from x_i and z
- Prior $p(z)$ can optionally depend on x

Application: multimodal imitation learning

- $p(\mathbf{a}|\mathbf{s},z)$ with $z \sim \mathcal{N}(0,\mathbf{I})$
- Different z values $\rightarrow$ different strategies/modes
- Examples: Learning Latent Plans from Play (Lynch et al.), ACT (Zhao et al.)

Connection to Diffusion and Flow Matching

Conditional VAE:

- Single latent $z \sim \mathcal{N}(0, \mathbf{I})$
- $p(\mathbf{a}|\mathbf{s}, z)$: one-step decode

$$\log p(\mathbf{a}|\mathbf{s}) \geq E_{z \sim q(z|\mathbf{s},\mathbf{a})}[\log p(\mathbf{a}|\mathbf{s}, z)]$$
$$- D_{\mathrm{KL}}(q(z|\mathbf{s}, \mathbf{a}), p(z))$$

Diffusion / flow matching:

- Hierarchical latent variables $a_{\Delta\tau:1}$
- Multi-step denoising chain

$$\log p(a_0|\mathbf{s}) \geq E_{a_{\Delta\tau:1} \sim q}[\log p(a_0|\mathbf{s}, a_{\Delta\tau:1})]$$
$$- D_{\mathrm{KL}}(q(a_{\Delta\tau:1}|a_0), p(a_{\Delta\tau:1}))$$

Diffusion can be thought of as a **hierarchical VAE**

State Space Models

VAE structure for sequential data:

- Latent: $\mathbf{z} = (\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_T)$
- Observations: $\mathbf{x} = (\mathbf{o}_1, \mathbf{o}_2, \ldots, \mathbf{o}_T)$

What is the prior?

$$p(\mathbf{z}) = p(\mathbf{z}_1) \prod_t p(\mathbf{z}_{t+1}|\mathbf{z}_t, \mathbf{a}_t)$$

What is the decoder?

$$p_\theta(\mathbf{o}|\mathbf{z}) = \prod_t p(\mathbf{o}_t|\mathbf{z}_t)$$

What is the encoder?

$$q_\phi(\mathbf{z}|\mathbf{o}) = \prod_t q_\phi(\mathbf{z}_t|\mathbf{o}_{1:t})$$

- Prior is *learned* (dynamics model)
- Connects to model-based RL!

Optimal Control as a Model of Human Behavior

Standard optimal control:

$$\mathbf{a}_1, \ldots, \mathbf{a}_T = \arg\max_{\mathbf{a}_1, \ldots, \mathbf{a}_T} \sum_{t=1}^{T} r(\mathbf{s}_t, \mathbf{a}_t)$$

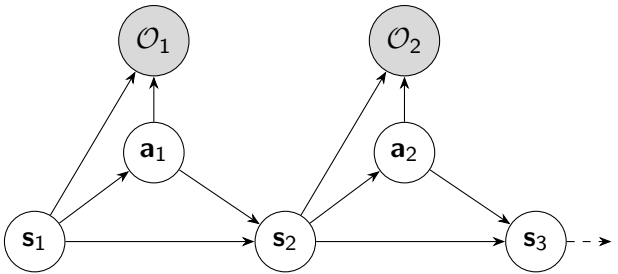
- Assumes behavior is **deterministic** and **optimal**
- But real behavior is **stochastic**
- Some mistakes matter more than others
- Good behavior is *more likely*, not guaranteed

Can we build a probabilistic model that explains this?

A Probabilistic Graphical Model of Decision Making

Introduce **optimality variables** $\mathcal{O}_t \in \{0, 1\}$:

$$p(\mathcal{O}_t = 1 \mid \mathbf{s}_t, \mathbf{a}_t) = \exp\big(r(\mathbf{s}_t, \mathbf{a}_t)\big) \qquad \text{(requires } r \leq 0\text{: subtract } \max r\text{; policy unchanged)}$$



$$p(\tau|\mathcal{O}_{1:T}) \propto p(\tau) \prod_t \exp\big(r(\mathbf{s}_t, \mathbf{a}_t)\big) = p(\tau) \exp\left(\sum_t r(\mathbf{s}_t, \mathbf{a}_t)\right)$$

Why Is This Interesting?

The graphical model perspective gives us:

1. Can model **suboptimal behavior** (important for inverse RL)
2. Can apply **inference algorithms** to solve control and planning problems
3. Provides an explanation for why **stochastic behavior** might be preferred
   - Useful for exploration and transfer learning

Control = inference in the graphical model with optimality variables

Inference = Planning

Three-step inference procedure:

1. Compute **backward messages** $\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_{t:T}|\mathbf{s}_t, \mathbf{a}_t)$
2. Compute **policy** $p(\mathbf{a}_t|\mathbf{s}_t, \mathcal{O}_{1:T})$
3. Compute **forward messages** $\alpha_t(\mathbf{s}_t) = p(\mathbf{s}_t|\mathcal{O}_{1:t-1})$

This is analogous to the **forward-backward algorithm** in HMMs!

Backward Messages

$\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_{t:T}|\mathbf{s}_t, \mathbf{a}_t) =$ probability of being optimal from t to T

Recursion (for $t = T-1$ to 1):

$$\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_t|\mathbf{s}_t, \mathbf{a}_t)\, E_{\mathbf{s}_{t+1}\sim p(\mathbf{s}_{t+1}|\mathbf{s}_t,\mathbf{a}_t)}\big[\beta_{t+1}(\mathbf{s}_{t+1})\big]$$

$$\beta_t(\mathbf{s}_t) = E_{\mathbf{a}_t\sim p(\mathbf{a}_t|\mathbf{s}_t)}\big[\beta_t(\mathbf{s}_t, \mathbf{a}_t)\big]$$

Base case: $\beta_T(\mathbf{s}_T, \mathbf{a}_T) = p(\mathcal{O}_T|\mathbf{s}_T, \mathbf{a}_T) = \exp\big(r(\mathbf{s}_T, \mathbf{a}_T)\big)$

Backward Pass $\rightarrow$ Soft Value Functions

Define log-space quantities:

$$V_t(\mathbf{s}_t) = \log \beta_t(\mathbf{s}_t), \qquad Q_t(\mathbf{s}_t, \mathbf{a}_t) = \log \beta_t(\mathbf{s}_t, \mathbf{a}_t)$$

Soft Bellman equations:

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \log E\big[\exp\big(V_{t+1}(\mathbf{s}_{t+1})\big)\big]$$

$$V_t(\mathbf{s}_t) = \log \int \exp\big(Q_t(\mathbf{s}_t, \mathbf{a}_t)\big)\, d\mathbf{a}_t$$

- $V_t(\mathbf{s}_t) \rightarrow \max_{\mathbf{a}_t} Q_t(\mathbf{s}_t, \mathbf{a}_t)$ as rewards get larger ("soft max")
- Deterministic transitions: $Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + V_{t+1}(\mathbf{s}_{t+1})$
- Stochastic transitions: $\log E[\exp V_{t+1}] \geq E[V_{t+1}]$ (Jensen), so the backup is **optimistic**, as if the agent could pick its next state (fixed by variational inference)
- Compare with standard value iteration: $Q = r + \gamma\, E[V']$, $V = \max_a Q$

The Action Prior

What if the action prior $p(\mathbf{a}_t|\mathbf{s}_t)$ is **not uniform**?

Modified soft value function:

$$V(\mathbf{s}_t) = \log \int \exp\big(Q(\mathbf{s}_t, \mathbf{a}_t) + \log p(\mathbf{a}_t|\mathbf{s}_t)\big)\, d\mathbf{a}_t$$

Define $\tilde{Q}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \log p(\mathbf{a}_t|\mathbf{s}_t) + \log E[\exp(V(\mathbf{s}_{t+1}))]$

Key insight: can *always* fold the action prior into the reward!

$$V(\mathbf{s}_t) = \log \int \exp\big(\tilde{Q}(\mathbf{s}_t, \mathbf{a}_t)\big)\, d\mathbf{a}_t \;\Leftrightarrow\; V(\mathbf{s}_t) = \log \int \exp\big(Q(\mathbf{s}_t, \mathbf{a}_t) + \log p(\mathbf{a}_t|\mathbf{s}_t)\big)\, d\mathbf{a}_t$$

Uniform action prior can be assumed WLOG.

Policy Computation

Compute $p(\mathbf{a}_t|\mathbf{s}_t, \mathcal{O}_{1:T}) = \pi(\mathbf{a}_t|\mathbf{s}_t)$:

$$\pi(\mathbf{a}_t|\mathbf{s}_t) = \frac{\beta_t(\mathbf{s}_t, \mathbf{a}_t)}{\beta_t(\mathbf{s}_t)} = \frac{\exp\big(Q_t(\mathbf{s}_t, \mathbf{a}_t)\big)}{\exp\big(V_t(\mathbf{s}_t)\big)}$$

Therefore:

$$\boxed{\pi(\mathbf{a}_t|\mathbf{s}_t) = \exp\big(Q_t(\mathbf{s}_t, \mathbf{a}_t) - V_t(\mathbf{s}_t)\big) = \exp\big(A_t(\mathbf{s}_t, \mathbf{a}_t)\big)}$$

where $A_t = Q_t - V_t$ is the **advantage function**.

With temperature α:

$$\pi(\mathbf{a}_t|\mathbf{s}_t) = \exp\left(\frac{1}{\alpha} A_t(\mathbf{s}_t, \mathbf{a}_t)\right)$$

- Better actions $\rightarrow$ higher probability
- $\alpha \rightarrow 0$: approaches greedy (deterministic) policy
- Analogous to Boltzmann exploration

Forward Messages

$\alpha_t(\mathbf{s}_t) = p(\mathbf{s}_t|\mathcal{O}_{1:t-1})$

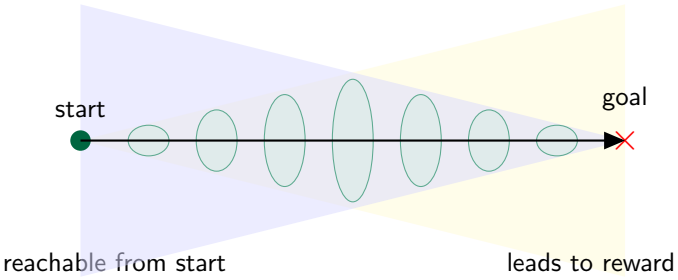
- $\alpha_1(\mathbf{s}_1) = p(\mathbf{s}_1)$ (initial state distribution, usually known)
- Recursion needs no β: $\alpha_{t+1}(\mathbf{s}_{t+1}) \propto \iint p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)\, e^{r(\mathbf{s}_t, \mathbf{a}_t)}\, p(\mathbf{a}_t|\mathbf{s}_t)\, \alpha_t(\mathbf{s}_t)\, d\mathbf{a}_t\, d\mathbf{s}_t$

State marginal:

$$p(\mathbf{s}_t|\mathcal{O}_{1:T}) \propto \beta_t(\mathbf{s}_t)\, \alpha_t(\mathbf{s}_t)$$

- β_t: states with high probability of *reaching the goal* (backward)
- α_t: states with high probability of *being reached from start* (forward)
- Intersection: states that are both reachable **and** lead to high reward

Forward/Backward Message Intersection



$$p(\mathbf{s}_t) \propto \beta_t(\mathbf{s}_t)\,\alpha_t(\mathbf{s}_t) \quad \text{(state marginals)}$$

Summary

Part 1: Variational Inference

- Latent variable models: $p(x) = \int p(x|z)\, p(z)\, dz$
- ELBO: $\log p(x_i) \geq \mathcal{L}_i(p, q_i)$
- Variational identity: $\log p(x_i) = D_{\mathrm{KL}}(q_i \| p(z|x_i)) + \mathcal{L}_i$

Part 2: Amortized VI & VAEs

- Amortized inference: $q_\phi(z|x)$ replaces per-sample q_i
- Reparameterization trick: $z = \mu_\phi(x) + \epsilon\, \sigma_\phi(x)$
- Applications: representation learning, multimodal policies, diffusion

Part 3: Control as Inference

- Optimality variables: $p(\mathcal{O}_t|\mathbf{s}_t, \mathbf{a}_t) = \exp(r(\mathbf{s}_t, \mathbf{a}_t))$
- Soft Bellman equations: $V = \text{soft-max}_{\mathbf{a}}\, Q$
- Policy: $\pi(\mathbf{a}|\mathbf{s}) = \exp(A(\mathbf{s}, \mathbf{a}))$